

Verificación de Programas Concurrentes Usando Lógica

Logical Foundations of Concurrent Computation

Jorge A. Pérez
University of Groningen, The Netherlands
j.a.perez [[at]] rug.nl

ECI 2026
(Version of July 29, 2026)

Asynchronous Communication

Outline

Up to Here

Classical LL and Classical Processes

Asynchronous Processes

AP: A Basic Language

Asynchronous Processes with DF

Asynchronous CP

Priorities for AP

Up to Here

- ▶ Concurrent processes, session types, Linear Logic (LL).
- ▶ Concurrent interpretation of LL: statics and dynamics.
- ▶ Left and right rules give a **rely/guarantee** reading, per connective:
 - The right rule says how to **offer** a behavior specified by the connective;
 - The left rule says how to **use** such a behavior.
- ▶ Cut reductions and process synchronizations.
- ▶ The correctness properties ensured by the logic-based type system, in particular deadlock freedom (DF).
- ▶ The computational interpretation of proof transformations (reduction, but also structural congruence and commuting conversions).

Deadlock Freedom

- The type system derived from LL avoids deadlocks!

Deadlock Freedom

- The type system derived from LL avoids deadlocks!
- An example of a stuck process (different colors = different protocols):

$$P = (\nu x)(\nu z)(\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\bar{\textcolor{blue}{z}}\langle \textcolor{blue}{w} \rangle.(P_1 \mid P_2) \\ \mid (\nu y)\textcolor{blue}{z}(\textcolor{blue}{u}).(\bar{\textcolor{red}{x}}\langle \textcolor{red}{y} \rangle.P_3 \mid P_4))$$

A circular dependency:

the **blue output** **blocks** the **red input**; also,

Deadlock Freedom

- The type system derived from LL avoids deadlocks!
- An example of a stuck process (different colors = different protocols):

$$P = (\nu x)(\nu z)(\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\bar{\textcolor{blue}{z}}\langle \textcolor{blue}{w} \rangle.(P_1 \mid P_2) \\ \mid (\nu y)\textcolor{blue}{z}(\textcolor{blue}{u}).(\bar{\textcolor{red}{x}}\langle \textcolor{red}{y} \rangle.P_3 \mid P_4))$$

A circular dependency:

the **blue output** **blocks** the **red input**; also, the **blue input** **blocks** the **red output**.

Deadlock Freedom

- The type system derived from LL avoids deadlocks!
- An example of a stuck process (different colors = different protocols):

$$P = (\nu x)(\nu z)(\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\bar{\textcolor{blue}{z}}\langle \textcolor{blue}{w} \rangle.(P_1 \mid P_2) \\ \mid (\nu y)\textcolor{blue}{z}(\textcolor{blue}{u}).(\bar{\textcolor{red}{x}}\langle \textcolor{red}{y} \rangle.P_3 \mid P_4))$$

A circular dependency:

the **blue output blocks** the **red input**; also, the **blue input blocks** the **red output**.

- A similar process without this circular dependency:

$$Q = (\nu x)(\nu z)(\underbrace{\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\bar{\textcolor{blue}{z}}\langle \textcolor{blue}{w} \rangle.(Q_1 \mid Q_2)}_L \mid \underbrace{(\nu y)\bar{\textcolor{red}{x}}\langle \textcolor{red}{y} \rangle.(\textcolor{blue}{z}(\textcolor{blue}{u}).Q_3 \mid Q_4)}_R)$$

Still, Q cannot be typed: .

Deadlock Freedom

- The type system derived from LL avoids deadlocks!
- An example of a stuck process (different colors = different protocols):

$$P = (\nu x)(\nu z)(\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\textcolor{blue}{z}\langle \textcolor{blue}{w} \rangle.(P_1 \mid P_2) \\ \mid (\nu y)\textcolor{blue}{z}(\textcolor{blue}{u}).(\textcolor{red}{x}\langle \textcolor{red}{y} \rangle.P_3 \mid P_4))$$

A circular dependency:

the **blue output blocks** the **red input**; also, the **blue input blocks** the **red output**.

- A similar process without this circular dependency:

$$Q = (\nu x)(\nu z)(\underbrace{\textcolor{red}{x}(\textcolor{red}{y}).(\nu w)\textcolor{blue}{z}\langle \textcolor{blue}{w} \rangle.(Q_1 \mid Q_2)}_L \mid \underbrace{(\nu y)\textcolor{red}{x}\langle \textcolor{red}{y} \rangle.(z(\textcolor{blue}{u}).Q_3 \mid Q_4)}_R)$$

Still, Q cannot be typed: sub-processes L and R share **two** channels (x and z).

Two Communication Mechanisms

- ▶ Our process model has **blocking actions** to induce sequential patterns specified by protocols. Deadlocks arise from issues in how actions block each other.

Two Communication Mechanisms

- ▶ Our process model has **blocking actions** to induce sequential patterns specified by protocols. Deadlocks arise from issues in how actions block each other.
- ▶ We have worked in a **synchronous** setting, where both outputs and inputs are blocking:

$$\overline{x}\langle z \rangle.P \mid x(y).Q \longrightarrow P \mid Q\{z/y\}$$

Before reduction, P and Q are blocked; after reduction, they are enabled.

Two Communication Mechanisms

- ▶ Our process model has **blocking actions** to induce sequential patterns specified by protocols. Deadlocks arise from issues in how actions block each other.
- ▶ We have worked in a **synchronous** setting, where both outputs and inputs are blocking:

$$\bar{x}\langle z \rangle.P \mid x(y).Q \longrightarrow P \mid Q\{z/y\}$$

Before reduction, P and Q are blocked; after reduction, they are enabled.

- ▶ In an **asynchronous** setting, communication is simpler—only inputs are blocking:

$$\bar{x}\langle z \rangle.0 \mid x(y).Q \longrightarrow Q\{z/y\}$$

The output does not block any actions (only 0), the actions in Q are enabled after reduction.

The Essentials of DF

We consider a simpler setting to investigate DF, in two steps:

1. We move to the concurrent interpretation of **classical** linear logic (CLL): the same expressive power as ILL, with less rules.¹

¹Philosophical considerations apply.

The Essentials of DF

We consider a simpler setting to investigate DF, in two steps:

1. We move to the concurrent interpretation of **classical** linear logic (CLL): the same expressive power as ILL, with less rules.¹
2. We move to a process model with **asynchronous** communication, equipped with a continuation-passing mechanism to express sequential protocols.

¹Philosophical considerations apply.

Outline

Up to Here

Classical LL and Classical Processes

Asynchronous Processes

AP: A Basic Language

Asynchronous Processes with DF

Asynchronous CP

Priorities for AP

Classical Linear Logic / CP

Propositions and their Concurrent Interpretation (Wadler's Classical Processes):

$A, B ::= A \otimes B$	send
$A \wp B$	receive
$\oplus \{i : A\}_{i \in I}$	selection
$\& \{i : A\}_{i \in I}$	branching
1	closed protocol
$\perp$	closed protocol

Classical Linear Logic

The **dual** of a proposition (session type) A , denoted $\overline{A}$:

$$\overline{A \otimes B} \triangleq \overline{A} \wp \overline{B}$$

$$\overline{A \wp B} \triangleq \overline{A} \otimes \overline{B}$$

$$\overline{1} \triangleq \perp$$

$$\overline{\oplus\{i : A_i\}_{i \in I}} \triangleq \&\{i : \overline{A_i}\}_{i \in I}$$

$$\overline{\&\{i : A_i\}_{i \in I}} \triangleq \oplus\{i : \overline{A_i}\}_{i \in I}$$

$$\overline{\perp} \triangleq 1$$

The Process Language (Synchronous)

$P, Q ::=$	$[y \leftarrow x]$	forwarder between sessions x and y
	$(\nu y) \bar{x}\langle y \rangle.(P \mid Q)$	send y over x , then execute P and Q
	$x(y).P$	receive y over x , then execute P
	$\bar{x}\langle \rangle$	close session x
	$x().P$	wait-close session x , then execute P
	$x \triangleleft 1_i; P$	select label 1_i along x , then execute P
	$x \triangleright \{1_1 : P_1, \dots, 1_n : P_n\}$	branch on x , offering labels $1_1, \dots, 1_n$
	0	inaction
	$(\nu xy)(P \mid Q)$	parallel : x and y are dual endpoints in P

The Process Language (Synchronous)

$P, Q ::=$	
$[y \leftarrow x]$	forwarder between sessions x and y
$(\nu y) \bar{x}\langle y \rangle.(P \mid Q)$	send y over x , then execute P and Q
$x(y).P$	receive y over x , then execute P
$\bar{x}\langle \rangle$	close session x
$x().P$	wait-close session x , then execute P
$x \triangleleft 1_i; P$	select label 1_i along x , then execute P
$x \triangleright \{1_1 : P_1, \dots, 1_n : P_n\}$	branch on x , offering labels $1_1, \dots, 1_n$
0	inaction
$(\nu xy)(P \mid Q)$	parallel : x and y are dual endpoints in P

The Process Language (Synchronous)

$P, Q ::=$	
$[y \leftarrow x]$	forwarder between sessions x and y
$(\nu y) \bar{x}\langle y \rangle.(P \mid Q)$	send y over x , then execute P and Q
$x(y).P$	receive y over x , then execute P
$\bar{x}\langle \rangle$	close session x
$x().P$	wait-close session x , then execute P
$x \triangleleft 1_i; P$	select label 1_i along x , then execute P
$x \triangleright \{1_1 : P_1, \dots, 1_n : P_n\}$	branch on x , offering labels $1_1, \dots, 1_n$
0	inaction
$(\nu xy)(P \mid Q)$	parallel : x and y are dual endpoints in P

A useful notation: We write $\bar{x}(y).(P \mid Q)$ instead of $(\nu y)\bar{x}\langle y \rangle.(P \mid Q)$.

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \longrightarrow (\nu uw)(P \mid (\nu xy)(Q \mid R))}$$

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \longrightarrow (\nu uw)(P \mid (\nu xy)(Q \mid R))}$$

[red-sel-bra]

$$\frac{j \in I}{(\nu xy)(x \triangleleft 1_j; P \mid y \triangleright \{1_1 : P_1, \dots, 1_n : P_n\}) \longrightarrow (\nu xy)(P \mid Q_j)}$$

[red-fwd]

$$\frac{y \neq z}{(\nu xy)([x \leftarrow z] \mid P) \longrightarrow P\{z/y\}}$$

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \longrightarrow (\nu uw)(P \mid (\nu xy)(Q \mid R))}$$

[red-sel-bra]

$$\frac{j \in I}{(\nu xy)(x \triangleleft 1_j; P \mid y \triangleright \{1_1 : P_1, \dots, 1_n : P_n\}) \longrightarrow (\nu xy)(P \mid Q_j)}$$

[red-fwd]

$$\frac{y \neq z}{(\nu xy)([x \leftarrow z] \mid P) \longrightarrow P\{z/y\}}$$

[red-sc]

$$\frac{P \equiv P' \quad P' \longrightarrow P' \quad Q' \equiv Q}{P \longrightarrow Q}$$

[red-res]

$$\frac{P \longrightarrow Q}{(\nu xy)P \longrightarrow (\nu xy)Q}$$

[red-par]

$$\frac{P \longrightarrow Q}{P \mid R \longrightarrow Q \mid R}$$

Typing Judgments

Judgments are of the form

$$P \vdash \Gamma$$

where

- ▶ P is a process
- ▶ Γ records endpoint/protocol assignments of the form $x : A$.

Typing Judgments

Judgments are of the form

$$P \vdash \Gamma$$

where

- ▶ P is a process
- ▶ Γ records endpoint/protocol assignments of the form $x : A$.

The context Γ disallows

- ▶ *weakening* (all assignments must be used)
- ▶ *contraction* (assignments may not be duplicated).

but obeys *exchange* (assignments may be silently reordered).

The empty context is written $\emptyset$.

Typing Rules based on CLL

[typ-fwd]

$$\frac{}{[x \leftarrow y] \vdash x : \overline{A}, y : A}$$

Typing Rules based on CLL

[typ-fwd]

$$\frac{[x \leftarrow y] \vdash x : \overline{A}, y : A}{[x \leftarrow y] \vdash x : \overline{A}, y : A}$$

[typ-cut]

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta}$$

Typing Rules based on CLL

[typ-fwd]

$$\frac{}{[x \leftarrow y] \vdash x : \overline{A}, y : A}$$

[typ-cut]

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta}$$

[typ-send]

$$\frac{P \vdash \Gamma, y : A \quad Q \vdash \Delta, x : B}{\overline{x}(y).(P \mid Q) \vdash \Gamma, \Delta, x : A \otimes B}$$

Typing Rules based on CLL

[typ-fwd]

$$\frac{}{[x \leftarrow y] \vdash x : \overline{A}, y : A}$$

[typ-send]

$$\frac{P \vdash \Gamma, y : A \quad Q \vdash \Delta, x : B}{\overline{x}(y).(P \mid Q) \vdash \Gamma, \Delta, x : A \otimes B}$$

[typ-cut]

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, x : B}{x(y).P \vdash \Gamma, x : A \wp B}$$

Typing Rules based on CLL

[typ-fwd]

$$\frac{}{[x \leftarrow y] \vdash x : \bar{A}, y : A}$$

[typ-cut]

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \bar{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta}$$

[typ-send]

$$\frac{P \vdash \Gamma, y : A \quad Q \vdash \Delta, x : B}{\bar{x}(y).(P \mid Q) \vdash \Gamma, \Delta, x : A \otimes B}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, x : B}{x(y).P \vdash \Gamma, x : A \wp B}$$

[typ-sel]

$$\frac{P \vdash \Gamma, x : A_j \quad j \in I}{x \triangleleft 1_j; P \vdash x : \oplus \{i : A_i\}_{i \in I}}$$

[typ-bra]

$$\frac{\forall i \in I : P_i \vdash \Gamma, x : A_i}{y \triangleright \{1_1 : P_1, \dots, 1_n : P_n\} \vdash \Gamma, x : \& \{i : A_i\}_{i \in I}}$$

[typ-inact]

$$\frac{}{0 \vdash \emptyset}$$

Typing Rules based on CLL

[typ-fwd]

$$\frac{}{[x \leftarrow y] \vdash x : \overline{A}, y : A}$$

[typ-cut]

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta}$$

[typ-send]

$$\frac{P \vdash \Gamma, y : A \quad Q \vdash \Delta, x : B}{\overline{x}(y).(P \mid Q) \vdash \Gamma, \Delta, x : A \otimes B}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, x : B}{x(y).P \vdash \Gamma, x : A \wp B}$$

[typ-sel]

$$\frac{P \vdash \Gamma, x : A_j \quad j \in I}{x \triangleleft 1_j; P \vdash x : \oplus \{i : A_i\}_{i \in I}}$$

[typ-bra]

$$\frac{\forall i \in I : P_i \vdash \Gamma, x : A_i}{y \triangleright \{1_1 : P_1, \dots, 1_n : P_n\} \vdash \Gamma, x : \& \{i : A_i\}_{i \in I}}$$

[typ-inact]

$$\frac{}{0 \vdash \emptyset}$$

[typ-one]

$$\frac{}{\overline{x} \langle \rangle \vdash \Gamma, x : 1}$$

[typ-bot]

$$\frac{P \vdash \Gamma}{x().P \vdash \Gamma, x : \perp}$$

Example: Principal Cut Reduction for $\otimes/\wp$

$$\frac{\frac{P \vdash \Gamma_1, u : A \quad Q \vdash \Gamma_2, x : B}{\bar{x}(u).(P \mid Q) \vdash \Gamma_1, \Gamma_2, x : A \otimes B} \quad \frac{R \vdash \Delta, w : \bar{A}, y : \bar{B}}{y(w).R \vdash \Delta, y : \bar{A} \wp \bar{B}}}{(\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \vdash \Gamma_1, \Gamma_2, \Delta}$$

$\longrightarrow$

Example: Principal Cut Reduction for $\otimes/\wp$

$$\begin{array}{c}
 \frac{P \vdash \Gamma_1, u : A \quad Q \vdash \Gamma_2, x : B}{\bar{x}(u).(P \mid Q) \vdash \Gamma_1, \Gamma_2, x : A \otimes B} \quad \frac{R \vdash \Delta, w : \bar{A}, y : \bar{B}}{y(w).R \vdash \Delta, y : \bar{A} \wp \bar{B}} \\
 \hline
 (\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \vdash \Gamma_1, \Gamma_2, \Delta \\
 \\
 \longrightarrow \\
 \frac{Q \vdash \Gamma_2, x : B \quad R \vdash \Delta, w : \bar{A}, y : \bar{B}}{} \\
 \hline
 \end{array}$$

Example: Principal Cut Reduction for $\otimes/\wp$

$$\begin{array}{c}
 \frac{P \vdash \Gamma_1, u : A \quad Q \vdash \Gamma_2, x : B}{\bar{x}(u).(P \mid Q) \vdash \Gamma_1, \Gamma_2, x : A \otimes B} \quad \frac{R \vdash \Delta, w : \bar{A}, y : \bar{B}}{y(w).R \vdash \Delta, y : \bar{A} \wp \bar{B}} \\
 \hline
 (\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \vdash \Gamma_1, \Gamma_2, \Delta \\
 \\
 \longrightarrow \\
 \frac{Q \vdash \Gamma_2, x : B \quad R \vdash \Delta, w : \bar{A}, y : \bar{B}}{(\nu xy)(Q \mid R) \vdash \Gamma_2, \Delta, w : \bar{A}} \\
 \hline
 \end{array}$$

Example: Principal Cut Reduction for $\otimes/\wp$

$$\begin{array}{c}
 \frac{P \vdash \Gamma_1, u : A \quad Q \vdash \Gamma_2, x : B}{\bar{x}(u).(P \mid Q) \vdash \Gamma_1, \Gamma_2, x : A \otimes B} \quad \frac{R \vdash \Delta, w : \bar{A}, y : \bar{B}}{y(w).R \vdash \Delta, y : \bar{A} \wp \bar{B}} \\
 \hline
 (\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \vdash \Gamma_1, \Gamma_2, \Delta \\
 \\
 \longrightarrow \\
 \frac{P \vdash \Gamma_1, u : A \quad \frac{Q \vdash \Gamma_2, x : B \quad R \vdash \Delta, w : \bar{A}, y : \bar{B}}{(\nu xy)(Q \mid R) \vdash \Gamma_2, \Delta, w : \bar{A}}}{(\nu uw)(P \mid (\nu xy)(Q \mid R)) \vdash \Gamma_1, \Gamma_2, \Delta}
 \end{array}$$

Example: Principal Cut Reduction for $\otimes/\wp$

$$\begin{array}{c}
 \frac{P \vdash \Gamma_1, u : A \quad Q \vdash \Gamma_2, x : B}{\bar{x}(u).(P \mid Q) \vdash \Gamma_1, \Gamma_2, x : A \otimes B} \quad \frac{R \vdash \Delta, w : \bar{A}, y : \bar{B}}{y(w).R \vdash \Delta, y : \bar{A} \wp \bar{B}} \\
 \hline
 (\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \vdash \Gamma_1, \Gamma_2, \Delta \\
 \\
 \longrightarrow \\
 \frac{P \vdash \Gamma_1, u : A \quad \frac{Q \vdash \Gamma_2, x : B \quad R \vdash \Delta, w : \bar{A}, y : \bar{B}}{(\nu xy)(Q \mid R) \vdash \Gamma_2, \Delta, w : \bar{A}}}{(\nu uw)(P \mid (\nu xy)(Q \mid R)) \vdash \Gamma_1, \Gamma_2, \Delta}
 \end{array}$$

This way, we have **extracted** the reduction rule on processes:

$$(\nu xy)(\bar{x}(u).(P \mid Q) \mid y(w).R) \longrightarrow (\nu uw)(P \mid (\nu xy)(Q \mid R))$$

Outline

Up to Here

Classical LL and Classical Processes

Asynchronous Processes

AP: A Basic Language

Asynchronous Processes with DF

Asynchronous CP

Priorities for AP

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Next An overview of DF enforcement in three typed asynchronous π -calculi:

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Next An overview of DF enforcement in three typed asynchronous π -calculi:

1. AP: processes follow protocols but freely deadlock (no DF enforcement)

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Next An overview of DF enforcement in three typed asynchronous π -calculi:

1. AP: processes follow protocols but freely deadlock (no DF enforcement)
2. ACP: DF by typing for tree-like process topologies (basic enforcement)

Some Observations

- ▶ Asynchronous communication is the cleanest setting to study DF by typing
- ▶ Asynchrony unblocks behaviors / session types constrain behaviors
- ▶ Asynchronous sessions “in between” untyped synchrony and asynchrony
 - actions from different sessions are independent
 - actions of the same session should respect their session type

Next An overview of DF enforcement in three typed asynchronous π -calculi:

1. AP: processes follow protocols but freely deadlock (no DF enforcement)
2. ACP: DF by typing for tree-like process topologies (basic enforcement)
3. APCP: DF also for circular process topologies (advanced enforcement)

An Asynchronous Process Language

$P, Q ::= x[a, b]$	send: both a and b are free
$x(y, z); P$	receive
$P \mid Q$	parallel
$(\nu xy)P$	restriction
0	inaction
$[x \leftrightarrow y]$	forwarder
$\dots$	

An Asynchronous Process Language

$P, Q ::= x[a, b]$	send: both a and b are free
$x(y, z); P$	receive
$P \mid Q$	parallel
$(\nu xy)P$	restriction
0	inaction
$[x \leftrightarrow y]$	forwarder
$\dots$	

Conventions:

- ▶ $a, b, c, \dots, x, y, z, \dots$ denote **names** (or **endpoints**)
- ▶ Early letters of the alphabet denote **objects** of output-like constructs.
- ▶ $\tilde{x}, \tilde{y}, \tilde{z}, \dots$ denote finite sequences of names.

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(x[a, b] \mid y(a', b'); Q) \longrightarrow Q\{a/a', b/b'\}}$$

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(x[a, b] \mid y(a', b'); Q) \longrightarrow Q\{a/a', b/b'\}}$$

[red-sel-bra]

$$\frac{j \in I}{(\nu xy)(x[b] \triangleleft j \mid y(b') \triangleright \{i : Q_i\}_{i \in I}) \longrightarrow Q_j\{b/b'\}}$$

[red-fwd]

$$\frac{y \neq z}{(\nu xy)([x \leftrightarrow z] \mid P) \longrightarrow P\{z/y\}}$$

Reduction

[red-send-recv]

$$\frac{}{(\nu xy)(x[a, b] \mid y(a', b'); Q) \longrightarrow Q\{a/a', b/b'\}}$$

[red-sel-bra]

$$\frac{j \in I}{(\nu xy)(x[b] \triangleleft j \mid y(b') \triangleright \{i : Q_i\}_{i \in I}) \longrightarrow Q_j\{b/b'\}}$$

[red-fwd]

$$\frac{y \neq z}{(\nu xy)([x \leftrightarrow z] \mid P) \longrightarrow P\{z/y\}}$$

[red-sc]

$$\frac{P \equiv P' \quad P' \longrightarrow P' \quad Q' \equiv Q}{P \longrightarrow Q}$$

[red-res]

$$\frac{P \longrightarrow Q}{(\nu xy)P \longrightarrow (\nu xy)Q}$$

[red-par]

$$\frac{P \longrightarrow Q}{P \mid R \longrightarrow Q \mid R}$$

Derived Constructs / Syntactic Sugar

$$\overline{x}[y] \cdot P \triangleq (\nu ya)(\nu zb)(x[a, b] \mid P\{z/x\})$$

$$x(y); P \triangleq x(y, z); P\{z/x\}$$

$$\overline{x} \triangleleft \ell \cdot P \triangleq (\nu zb)(x[b] \triangleleft \ell \mid P\{z/x\})$$

$$x \triangleright \{i : P_i\}_{i \in I} \triangleq x(z) \triangleright \{i : P_i\{z/x\}\}_{i \in I}$$

Note: We use ‘ $\cdot$ ’ to signify that output-like operations are non blocking.

Continuations Induce Session Protocols

$$\begin{aligned} P \triangleq (\nu zu) \Big(& (\nu xy) \Big((\nu ax') (x[v_1, a] \mid x'[v_2, b]) \\ & \mid (\nu cz') (z[v_3, c] \mid y(w_1, y'); y'(w_2, y''); Q) \Big) \\ & \mid u(w_3, u'); R \Big) \end{aligned}$$

Continuations Induce Session Protocols

$$P \triangleq (\nu zu) \Big((\nu xy) \Big((\nu \textcolor{brown}{a} \textcolor{brown}{x}') (x[v_1, \textcolor{brown}{a}] \mid \textcolor{brown}{x}'[v_2, b]) \\ \mid (\nu cz') (z[v_3, c] \mid y(w_1, \textcolor{brown}{y}'); \textcolor{brown}{y}'(w_2, y''); Q) \Big) \\ \mid u(w_3, u'); R \Big)$$

Using a **sugared syntax**:

$$P = (\nu zu) ((\nu xy) (\bar{x}[v_1] \cdot \bar{x}[v_2] \cdot 0 \mid \bar{z}[v_3] \cdot y(w_1); y(w_2); Q') \mid u(w_3); R')$$

Continuations Induce Session Protocols

$$P \triangleq (\nu zu) \Big((\nu xy) \Big((\nu \textcolor{red}{ax}') (x[v_1, \textcolor{red}{a}] \mid \textcolor{red}{x}'[v_2, b]) \\ \mid (\nu cz') (z[v_3, c] \mid y(w_1, \textcolor{red}{y}'); \textcolor{red}{y}'(w_2, y''); Q) \Big) \\ \mid u(w_3, u'); R \Big)$$

Using a **sugared syntax**:

$$P = (\nu zu) ((\nu xy) (\bar{x}[v_1] \cdot \bar{x}[v_2] \cdot 0 \mid \bar{z}[v_3] \cdot y(w_1); y(w_2); Q') \mid u(w_3); R')$$

We have independent reductions:

$$P \longrightarrow (\nu zu) ((\nu xy) (\bar{x}[v_2] \cdot 0 \mid \bar{z}[v_3] \cdot y(w_2); Q'\{v_1/w_1\}) \mid u(w_3); R')$$

$$P \longrightarrow (\nu xy) (\bar{x}[v_1] \cdot \bar{x}[v_2] \cdot 0 \mid y(w_1); y(w_2); Q') \mid R'\{v_3/w_3\}$$

Note: There is no reduction involving the send on x' : because x' is connected to the continuation of the send on x , it is thus not (yet) paired with a dual receive.

A Deadlocked Process, Now Asynchronous

The **cyclic dependency** between the two threads, now in the asynchronous case:

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

Process is stuck:

- ▶ The input on x is waiting for the right output on y ...
- ▶ ...the output on y is blocked by a receive (on w) waiting for the left output on u , which is blocked by the input on x .

Session Types

Types specify protocols. They (mostly) correspond to Linear Logic propositions:

$$\begin{array}{lcl} A, B ::= A \otimes B & & \text{send} \\ | A \wp B & & \text{receive} \\ | \bullet & & \text{closed protocol} \\ | \dots & & \end{array}$$

Session Types

Types specify protocols. They (mostly) correspond to Linear Logic propositions:

$$\begin{array}{ll} A, B ::= A \otimes B & \text{send} \\ | A \wp B & \text{receive} \\ | \bullet & \text{closed protocol} \\ | \dots & \end{array}$$

The **dual** of session type A , denoted $\overline{A}$:

$$\begin{aligned} \overline{A \otimes B} &\triangleq \overline{A} \wp \overline{B} \\ \overline{A \wp B} &\triangleq \overline{A} \otimes \overline{B} \\ \overline{\bullet} &\triangleq \bullet \end{aligned}$$

Typing Judgments

Judgments are of the form

$$P \vdash \Gamma$$

where

- ▶ P is a process
- ▶ Γ records endpoint/protocol assignments of the form $x : A$.

Typing Judgments

Judgments are of the form

$$P \vdash \Gamma$$

where

- ▶ P is a process
- ▶ Γ records endpoint/protocol assignments of the form $x : A$.

The context Γ disallows

- ▶ *weakening* (all assignments must be used, except names typed with $\bullet$)
- ▶ *contraction* (assignments may not be duplicated).

but obeys *exchange* (assignments may be silently reordered).

The empty context is written $\emptyset$.

AP: Selected Typing Rules

[typ-send]

$$\frac{}{x[a, b] \vdash x : A \otimes B, a : \overline{A}, b : \overline{B}}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, z : B}{x(y, z); P \vdash \Gamma, x : A \wp B}$$

AP: Selected Typing Rules

[typ-send]

$$\frac{}{x[a, b] \vdash x : A \otimes B, a : \overline{A}, b : \overline{B}}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, z : B}{x(y, z); P \vdash \Gamma, x : A \wp B}$$

[typ-par]

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta}$$

[typ-res]

$$\frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma}$$

AP: Selected Typing Rules

[typ-send]

$$\frac{}{x[a, b] \vdash x : A \otimes B, a : \overline{A}, b : \overline{B}}$$

[typ-recv]

$$\frac{P \vdash \Gamma, y : A, z : B}{x(y, z); P \vdash \Gamma, x : A \wp B}$$

[typ-par]

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta}$$

[typ-res]

$$\frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma}$$

[typ-fwd]

$$\frac{}{[x \leftrightarrow y] \vdash x : \overline{A}, y : A}$$

[typ-end]

$$\frac{P \vdash \Gamma}{P \vdash \Gamma, x : \bullet}$$

[typ-inact]

$$\frac{}{0 \vdash \emptyset}$$

Properties of AP

Well-typed processes respect their session protocols under reduction:

Theorem (Type Preservation for AP)

Given $P \vdash \Gamma$ and Q such that $P \equiv Q$ or $P \longrightarrow Q$, we have $Q \vdash \Gamma$.

Deadlocks are Typable in AP

Recall the process

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

It can be typed as follows:

$$\begin{array}{c}
 \frac{}{u[a, b] \vdash u : \bullet \otimes \bullet, a : \bullet, b : \bullet} \\
 \hline
 u[a, b] \vdash u : \bullet \otimes \bullet, \\
 \quad v : \bullet, x' : \bullet, a : \bullet, b : \bullet \\
 \hline
 x(v, x'); u[a, b] \vdash x : \bullet \wp \bullet, \\
 \quad u : \bullet \otimes \bullet, \\
 \quad a : \bullet, b : \bullet \\
 \hline
 \vdots \\
 w(z, w'); y[c, d] \vdash w : \bullet \wp \bullet, \\
 \quad y : \bullet \otimes \bullet, \\
 \quad c : \bullet, d : \bullet \\
 \hline
 x(v, x'); u[a, b] \mid w(z, w'); y[c, d] \vdash x : \bullet \wp \bullet, u : \bullet \otimes \bullet, a : \bullet, b : \bullet, \\
 \quad w : \bullet \wp \bullet, y : \bullet \otimes \bullet, c : \bullet, d : \bullet \\
 \hline
 (\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d]) \vdash a : \bullet, b : \bullet, c : \bullet, d : \bullet
 \end{array}
 \begin{array}{l}
 \text{[typ-end]}^2 \\
 \\
 \text{[typ-recv]} \\
 \\
 \\
 \text{[typ-par]} \\
 \text{[typ-res]}^2
 \end{array}$$

Outline

Up to Here

Classical LL and Classical Processes

Asynchronous Processes

AP: A Basic Language

Asynchronous Processes with DF

Asynchronous CP

Priorities for AP

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).
- ▶ Obtained from AP by a minor yet crucial modification.
 - **Remove:**

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta} [\text{typ-par}] \qquad \frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma} [\text{typ-res}]$$

+ **Add:**

$$\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta} [\text{typ-cut}]$$

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).
- ▶ Obtained from AP by a minor yet crucial modification.
 - **Remove:**

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta} [\text{typ-par}] \qquad \frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma} [\text{typ-res}]$$

+ **Add:**

$$\boxed{\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta} [\text{typ-cut}]}$$

- ▶ ACP guarantees DF by ruling out all possible cyclic dependencies: sub-processes can connect along **exactly one pair of names**.

ACP: Asynchronous CP

- ▶ An asynchronous variant of Wadler's Classical Processes (CP).
- ▶ Obtained from AP by a minor yet crucial modification.
 - **Remove:**

$$\frac{P \vdash \Gamma \quad Q \vdash \Delta}{P \mid Q \vdash \Gamma, \Delta} [\text{typ-par}] \qquad \frac{P \vdash \Gamma, x : A, y : \overline{A}}{(\nu xy)P \vdash \Gamma} [\text{typ-res}]$$

+ **Add:**

$$\boxed{\frac{P \vdash \Gamma, x : A \quad Q \vdash \Delta, y : \overline{A}}{(\nu xy)(P \mid Q) \vdash \Gamma, \Delta} [\text{typ-cut}]}$$

- ▶ ACP guarantees DF by ruling out all possible cyclic dependencies: sub-processes can connect along **exactly one pair of names**.
- ▶ Note: We write $\textcolor{teal}{c}\vdash$ instead of $\vdash$ to distinguish the two type systems.

The Deadlocked Process, Revisited

- The deadlocked process

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

is not typable under $\textcolor{blue}{c}\vdash$:

its parallel sub-processes are connected on two pairs of names.

The Deadlocked Process, Revisited

- The deadlocked process

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

is not typable under $\textcolor{blue}{c}\vdash$:

its parallel sub-processes are connected on two pairs of names.

- A well-typed variant, obtained by ‘parallelizing’ the right sub-process:

$$(\nu xy)\big((\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); 0) \mid y[c, d]\big) \textcolor{blue}{c}\vdash \begin{array}{l} a : \bullet, b : \bullet, \\ c : \bullet, d : \bullet \end{array}$$

Properties of ACP

As in AP, well-typed processes respect their protocols:

Theorem (Type Preservation for ACP)

Given $P \text{ }^c\vdash \Gamma$ and Q such that $P \text{ }^c\equiv Q$ or $P \text{ }^c\longrightarrow Q$, we have $Q \text{ }^c\vdash \Gamma$.

Thanks to the cut rule, we now also have:

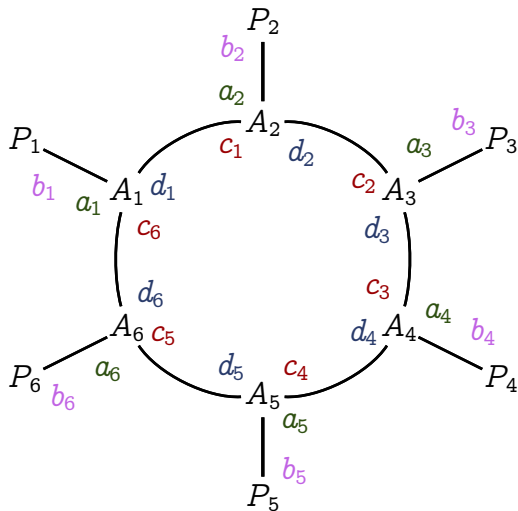
Theorem (Deadlock Freedom for ACP)

Given $P \text{ }^c\vdash \emptyset$, if $P \text{ }^c\nrightarrow$, then $P \text{ }^c\equiv 0$.

Not Typable in ACP: Milner's Cyclic Scheduler

Example of size 6:

$$\begin{aligned}
 &(\nu c_1 d_1) \cdots (\nu c_6 d_6) \Big((\nu a_1 b_1)(A_1 \mid P_1) \mid \\
 &\quad (\nu a_2 b_2)(A_2 \mid P_2) \mid \\
 &\quad \vdots \\
 &\quad (\nu a_6 b_6)(A_6 \mid P_6) \Big)
 \end{aligned}$$



APCP: Priority-Based DF Enforcement

Asynchronous **Priority-Based** Classical Processes:

- ▶ Back to **separate rules** for parallel composition and restriction (as in AP)
- ▶ Excluding cyclic dependencies using **priorities**

The Laws of Priorities

Key Idea

Typing ensures that each action has a corresponding priority (its 'urgency').
actions with **lower priority** must not be blocked by actions with **higher priority**.

The Laws of Priorities

Key Idea

Typing ensures that each action has a corresponding priority (its 'urgency').
actions with **lower priority** must not be blocked by actions with **higher priority**.

Write $\pi, \rho, \dots$ to denote priorities (integers). Laws enforced by typing:

1. Outputs with priority π send messages with **larger priority** than π ;
2. A action with priority π are prefixed by inputs with **smaller priority** than π ;
3. Dual actions must have **equal priorities**.

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

- We write ω to denote the ‘ultimate priority’: it is greater than all other priorities and cannot be increased further.

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

- We write ω to denote the ‘ultimate priority’: it is greater than all other priorities and cannot be increased further.
- For session type A , $pr(A)$ denotes its *priority*:

$$\begin{aligned} pr(A \otimes^{\pi} B) &\triangleq pr(A \wp^{\pi} B) \triangleq \pi \\ pr(\bullet) &\triangleq \omega \end{aligned}$$

APCP: Priority-Based DF Enforcement

- Types are as before, now annotated with priorities:

$$A, B ::= A \otimes^{\pi} B \mid A \wp^{\pi} B \mid \bullet \mid \dots$$

- We write ω to denote the ‘ultimate priority’: it is greater than all other priorities and cannot be increased further.
- For session type A , $pr(A)$ denotes its *priority*:

$$\begin{aligned} pr(A \otimes^{\pi} B) &\triangleq pr(A \wp^{\pi} B) \triangleq \pi \\ pr(\bullet) &\triangleq \omega \end{aligned}$$

- Duality: $A = \overline{B}$ iff (i) actions in A and B are complementary (as before) and (ii) their priorities coincide.

Typing rules of APCP: AP with Priorities

[typ-send]

$$\frac{\pi < pr(A), pr(B)}{x[y, z] \text{ }^P\vdash x : A \otimes^\pi B, y : \overline{A}, z : \overline{B}}$$

[typ-recv]

$$\frac{P \text{ }^P\vdash \Gamma, y : A, z : B \quad \pi < pr(\Gamma)}{x(y, z); P \text{ }^P\vdash \Gamma, x : A \wp^\pi B}$$

(Other rules similar or unchanged, with the extended duality)

Properties of APCP

Theorem (Type Preservation for APCP)

Given $P \vdash^P \Gamma$ and Q such that $P \equiv Q$ or $P \longrightarrow Q$, we have $Q \vdash^P \Gamma$.

Theorem (Deadlock Freedom for APCP)

Given $P \vdash^P \emptyset$, if $P \not\rightarrow$, then $P \equiv 0$.

Examples

- Recall the deadlocked process; it is rejected under $\textcolor{blue}{P} \vdash$:

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

Suppose that actions on x and y have priority $\textcolor{brown}{\pi}$, and that actions on u and w have priority $\textcolor{brown}{\rho}$. Using Rule [typ-recv] we require $\textcolor{brown}{\pi} < \textcolor{brown}{\rho}$ and $\textcolor{brown}{\rho} < \textcolor{brown}{\pi}$.

Examples

- Recall the deadlocked process; it is rejected under $\textcolor{blue}{P}\vdash$:

$$(\nu xy)(\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); y[c, d])$$

Suppose that actions on x and y have priority π , and that actions on u and w have priority ρ . Using Rule [typ-recv] we require $\pi < \rho$ and $\rho < \pi$.

- Recall the non-deadlocked variant:

$$(\nu xy)\big((\nu uw)(x(v, x'); u[a, b] \mid w(z, w'); 0) \mid y[c, d]\big)$$

In this case, $\textcolor{blue}{P}\vdash$ only requires $\pi < \rho$ but not $\rho < \pi$, so no cyclic dependency is detected—the process is considered well typed.

Application: DF in Concurrent Functional Sessions

LASTⁿ:
Concurrent functions
(can deadlock)

Application: DF in Concurrent Functional Sessions

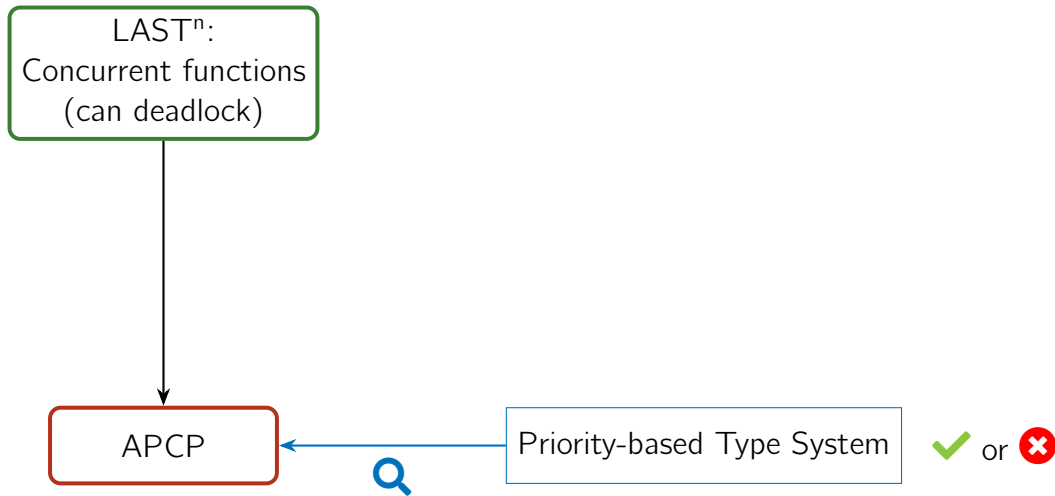
LASTⁿ:
Concurrent functions
(can deadlock)

APCP

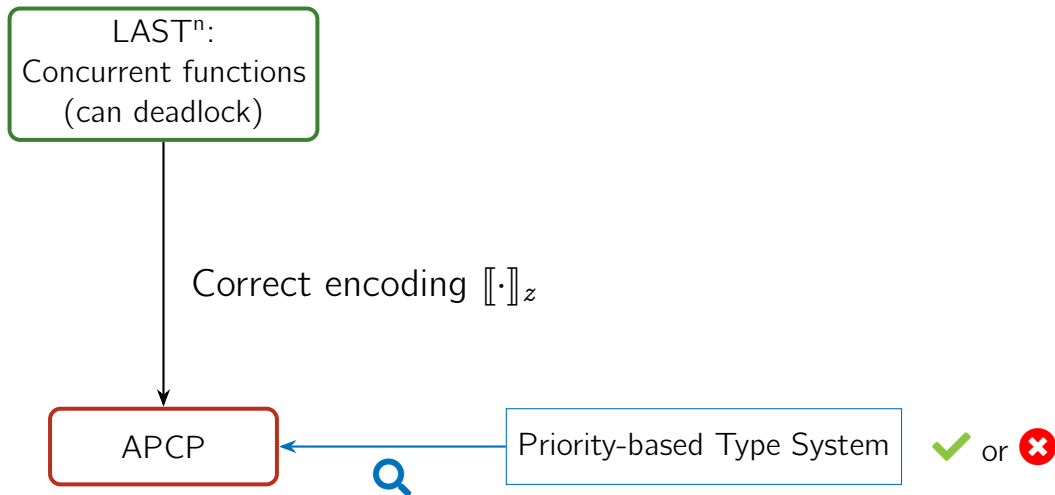
Priority-based Type System



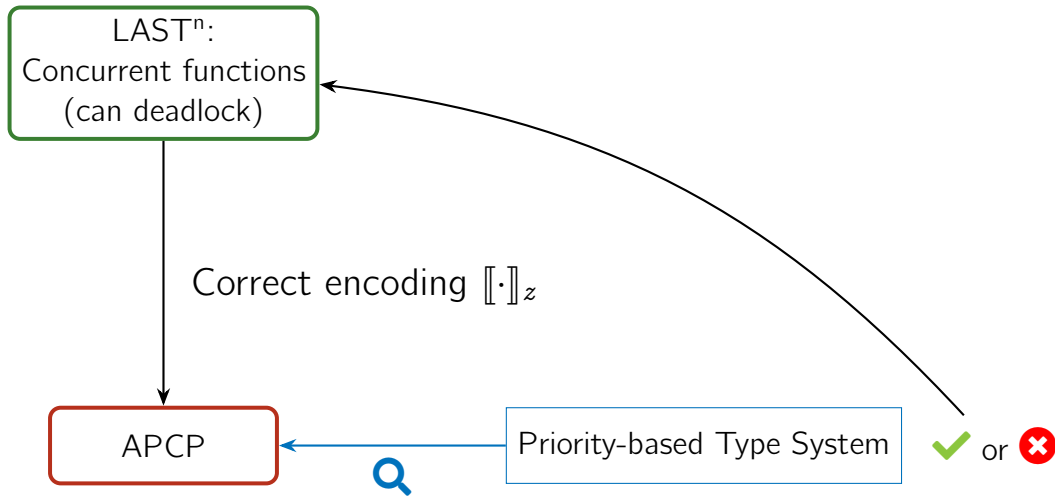
Application: DF in Concurrent Functional Sessions



Application: DF in Concurrent Functional Sessions



Application: DF in Concurrent Functional Sessions



Summing Up

The concurrent interpretation of Linear Logic, now in the classical presentation.

Three typed **asynchronous** π -calculi based on Linear Logic:

1. AP: processes correctly follow protocols but may deadlock
2. ACP: AP with logic-based composition, DF for tree-like topologies
3. APCP: extension of AP with priorities, DF for circular topologies